Porównanie tokenizerów: BPE, Whitespace i SentencePiece

Dla trzech tokenizerów trenuję ten sam mały model językowy na fragmencie tekstów z Polskiej Wikipedii.

1. Dane

Korpus to polska Wikipedia (SpeakLeash `plwiki`), pierwsze ok. 60 mln znaków. Co 10. dokument trafia do zbioru ewaluacyjnego, reszta do treningowego, więc oba zbiory są rozłączne, nie ma przecieku między nimi. W kodzie, dzielimy ten korpus na dwa pliki `plwiki_train.txt` i `plwiki_eval.txt`. Pierwszy zawiera ok. 200 tyś. linii tekstu, a drugi 21000.

2. Tokenizery

Tokenizator GPT-2 jest już wytrenowawny, a rozmiar jego słowwnika wynosi $N = 50257$ tokenów. Z tego wynika, że zarówno SentencePiece, jak i nasz własny tokenizator też powinien mieć taki sam rozmiar.

BPE. Pretrenowany tokenizer GPT-2 z HuggingFace, bez dotrenowania. Działa na bajtach, więc nie gubi informacji, ale jego słownik powstał głównie na angielskim.

```
bpe = AutoTokenizer.from_pretrained("gpt2")
```

Whitespace. Własna implementacja (`custom_tokenizer.py`). Tekst jest zamieniany na małe litery i dzielony na ciągi liter/cyfr oraz pojedyncze znaki interpunkcji. Słownik to `<UNK>` i 50 256 najczęstszych tokenów ze zbioru treningowego. Słowa spoza słownika dostają id `<UNK>`.

```
TOKEN_PATTERN = re.compile(r"\w+|[^\w\s]", re.UNICODE)

def tokenize(self, text):
    if self.lowercase:
        text = text.lower()
    return TOKEN_PATTERN.findall(text)

def encode(self, text):
    unk_id = self.token_to_id[UNK]
    return [self.token_to_id.get(t, unk_id) for t in self.tokenize(text)]
```

SentencePiece. Model unigramowy wytrenowany na zbiorze treningowym (próbka 300 tys. linii).

```
spm.SentencePieceTrainer.train(
    sentence_iterator=train_lines(),
    model_prefix="data/sp",
    vocab_size=50257,
    model_type="unigram",
```

```
    input_sentence_size=300_000,
    shuffle_input_sentence=True,
)
```

3. Model i trening

Model jest taki sam dla wszystkich tokenizerów, zmienia się tylko tokenizer. To dekoder typu GPT zbudowany z `nn.TransformerEncoder` z maską przyczynową, z embeddingami współdzielonymi z warstwą wyjściową.

parametr	wartość
parametry	30,0 mln
warstwy / głowice / wymiar	6 / 6 / 384
kontekst	256 tokenów
dropout	0,1
optymalizator	AdamW, lr 3e-4, weight decay 0,1
harmonogram lr	rozgrzewka 100 kroków, potem cosinus do 10% lr
batch	16 sekwencji (4096 tokenów na krok)
kroki	15 000 (61,4 mln tokenów)
ziarno	42

Czasy pochodzą z logów Slurma (`runs/*.out`). Łączny czas zawiera 61 ewaluacji na zbiorze ewaluacyjnym. Jedna ewaluacja trwa tym dłużej, im więcej tokenów ma ten zbiór, więc BPE traci na niej najwięcej. Czas samych kroków treningu jest u wszystkich podobny, bo każdy krok to 4096 tokenów.

tokenizer	tokeny treningowe	tokeny ewaluacyjne	czas łączny	jedna ewaluacja	same kroki
BPE	28 488 267	3 125 075	2641 s (44 min)	ok. 22 s	ok. 1300 s (22 min)
whitespace	9 488 374	1 038 403	1768 s (29 min)	ok. 8 s	ok. 1280 s (21 min)
SentencePiece	11 512 600	1 282 224	1890 s (32 min)	ok. 9 s	ok. 1340 s (22 min)

4. Sprzęt

- Trening: klaster PLGrid, partycja `plgrid-gpu-v100`, 1 × Tesla V100-SXM2 32 GB, 4 rdzenie CPU, 40 GB RAM, PyTorch 2.14.1 (CUDA 12.6).
- Ewaluacja i pomiary tokenizerów: komputer lokalny, GeForce GTX 1660 Ti 6 GB.

5. Wyniki

Perplexity

Liczymy perplexity w przeliczeniu na słowo lub znak, a nie na token. Różne tokenizery mogą dzielić ten sam tekst na różną liczbę tokenów, przez co strata na token nie jest bezpośrednio porównywalna.

Całkowitą stratę tekstu definiujemy jako $L = -\ln P(\text{tekst})$. Jeśli `nats` oznacza średnią stratę na token, a `len(ids)` liczbę tokenów, to całkowita strata wynosi:

$$L = \text{nats} \cdot \text{len}(\text{ids})$$

Aby obliczyć perplexity na słowo lub znak, dzielimy całkowitą stratę przez odpowiednio liczbę słów lub znaków, a następnie obliczamy eksponentę. Dzięki temu otrzymujemy miarę znormalizowaną względem jednostki tekstu, co ułatwia porównywanie modeli wykorzystujących różne tokenizery.

```
nats, _ = evaluate(model, ids, config, device, n_chars)
word_ppl = math.exp(nats * len(ids) / n_words)
char_ppl = math.exp(nats * len(ids) / n_chars)
```

tokenizer	perplexity na słowo	perplexity na znak
BPE	6 672	3,31
whitespace	221	2,08
SentencePiece	1 779	2,77

OOV (whitespace)

	OOV	wszystkich	%
wystąpienia słów	108 060	846 769	12,76%
unikalne słowa	72 292	115 583	62,55%

Częste słowa są w słowniku, więc wśród wystąpień OOV jest niewiele. Rzadkich form (fleksja, nazwiska, liczby) jest za to bardzo dużo i prawie wszystkie są spoza słownika.

Statystyki tokenizacji

Pomiar na całym zbiorze ewaluacyjnym (6,2 MB), czyli tekście spoza zbioru treningowego tokenizerów.

tokenizer	tokeny/słowo	tokeny/znak	tokeny/s	słowa jako 1 token
BPE	3,71	0,505	ok. 0,9 mln	5 712 (4,4%)
whitespace	1,23	0,168	ok. 2,2 mln	54 316 (42,2%)
SentencePiece	1,52	0,207	ok. 2,3 mln	32 512 (25,3%)

- "Słowa jako 1 token" to unikalne słowa z eval (ok. 130 tys.) kodowane jednym tokenem, nie `<UNK>`.
- Szybkość mierzę na pierwszym ok. 1 MB tekstu, linia po linii.

- Whitespace ma więcej niż 1 token na słowo, bo interpunkcja jest osobnym tokenem.

Przykłady

Trzy teksty po ok. 50 słów: fragment „Pana Tadeusza” (A), definicja fotosyntezy (B) i fragment o Krakowie (C).

tekst	tokenizer	tokeny/słowo	słowa kodowane bezpośrednio
A (poezja)	BPE	3,94	14%
	whitespace	1,22	61%
	SentencePiece	2,00	53%
B (definicja)	BPE	3,64	21%
	whitespace	1,15	94%
	SentencePiece	1,30	87%
C (artykuł)	BPE	3,49	23%
	whitespace	1,17	96%
	SentencePiece	1,30	89%

Fragment tekstu B (`chloroplastach, zawierających`):

```
BPE:           Ġchlor | op | last | ach | , | Ġz | aw | ier | aj | Ä | ħ |
cy | ch
whitespace:    <UNK> | , | zawierających
SentencePiece: ▁chloro | plast | ach | , | ▁zawierających
```

Podsumowanie

Granularność. BPE dzieli polskie słowa średnio na 3–4 fragmenty, często rozbijając polskie znaki diakrytyczne (ą, ę, ł, ś) na dwa tokeny bajtowe. SentencePiece, wytrenowany na polskim tekście, częściej zachowuje całe słowa lub dzieli je na mniejsze jednostki odpowiadające morfemom, np. chloro|plast|ach. Tokenizer whitespace przypisuje natomiast jedno tokenowe oznaczenie każdemu słowu znajdującemu się w słowniku.

OOV (out-of-vocabulary). Tokenizer whitespace nie ma mechanizmu obsługi nieznanych słów poza zastąpieniem ich tokenem . W rezultacie rzadkie wyrazy, takie jak chloroplastach czy zaciekłe, a także liczby, mogą zostać zamienione na , co prowadzi do bezpowrotnej utraty informacji. Problem ten jest widoczny również w wygenerowanym tekście:

```
albert einstein był stanem stałym <UNK> po tym, że pracował w zakładzie
<UNK>, a następnie jako <UNK>, z <UNK> <UNK> <UNK>. ...
```

BPE i SentencePiece mogą zawsze rozbić słowo na mniejsze części, kosztem dłuższej sekwencji.

Perplexity. Whitespace ma najniższą perplexity (221, 2,08), ale to wynik nieporównywalny z resztą. Model przewiduje tekst bez wielkich liter, a 12,8% słów zastępuje jednym łatwym tokenem <UNK>. Uczciwie porównywać można BPE z SentencePiece, bo oba rozróżniają wielkość liter i nie mają <UNK>. SentencePiece jest wyraźnie lepszy: 2,77 wobec 3,31 na znak.

Koszt. BPE ma 2,4–3 razy więcej tokenów na ten sam tekst. Sam trening trwa tyle samo (ok. 22 minuty, stały budżet tokenów), ale ewaluacja i tokenizacja kosztują więcej: 44 minuty łącznie wobec 29–32 i najwolniejsze kodowanie. SentencePiece jest kompromisem: sekwencje dłuższe niż we whitespace, ale bez <UNK>, i najlepsza perplexity wśród tokenizerów bez utraty informacji.